1. Interroger l'API

Il existe plusieurs façons pour interroger une API REST, un simple appel Ajax peut suffire si aucune données sensibles ou données d'authentification n'est nécessaire.

Cependant la solution la plus couramment utilisée repose sur la mise en place d'une session cUrl qui suit le principe suivant :

- Initialisation : **$ch = curl_init();**
- Configuration des options
- Exécution de la requête : **$reponse = curl_exec($ch);**
- Vérification des erreurs cUrl
- Vérification du code de statut http
- Fermeture de la session : **curl_close($ch);**
- Analyse de la réponse si besoin

2. Configuration des options

Les options vont permettre d'indiquer l'url de l'API, la méthode (POST,GET DELETE ....), les données transmise dans le corps de la requête et les données transmises dans l'entête de la requête (Content-type, Authorization)

Nous pouvons transmettre chaque option à l'aide de l'instruction curl_setopt :

```
curl_setopt($ch, CURLOPT_URL, $url);
```

Nous pouvons transmettre l'ensemble des options par l'instruction curl_setopt_array

```
$options = [
   CURLOPT_URL => $url,
   CURLOPT_RETURNTRANSFER => true,
];
curl_setopt_array($ch, $options);
```

CURLOPT_URL : Définit l'URL de la requête.
CURLOPT_RETURNTRANSFER : Si défini à true, la réponse sera retournée en tant que chaîne de caractères au lieu d'être affichée directement.

S'il faut inclure des en-têtes http il faut ajouter :

```
   CURLOPT_HTTPHEADER => [
      'Content-type: application/json',
      'Authorization: Bearer ' . $token,
      'User-Agent: verghote'
   ],
```

S'il faut préciser un 'User_Agent' (**obligatoire avec l'API GitHub)**, nous pouvons le faire dans CURLOPT_HTTPHEADER mais aussi plus simplement en utilisant l'option CURLOPT_USERAGENT ce qui s'avère plus simple et direct pour spécifier uniquement l'en-tête User-Agent.

```
    CURLOPT_USERAGENT => "verghote",
```

La méthode par défaut est GET, si vous devez utiliser la méthode POST, il faut définir l'option CURLOPT_POST à true et fournir les données à envoyer avec CURLOPT_POSTFIELDS.

```
$data = [
    'id' => $id,
    'nom' => $nom,
    'date' => $date,
    'distance' => $distance
];

// Configuration des options cURL
$options = [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Content-type: application/json',
        'Authorization: Bearer ' . $token
    ],
    CURLOPT_POST => true,
    CURLOPT_POSTFIELDS => json_encode($data)
];
curl_setopt_array($ch, $options);
```

Pour effectuer une requete DELETE ou PATCH ou PUT il faut ajouter l'option CURLOPT_CUSTOMREQUEST

```
    CURLOPT_CUSTOMREQUEST => 'DELETE',
```

Le délai d'attente par défaut de cURL est généralement de 300 secondes (5 minutes), il est possible de définir ce délai :

```
    CURLOPT_TIMEOUT => 30,
```

Si l'API nécessite une validation du certificat SSL il faut ajouter l'option CURLOPT_CAINFO

```
    CURLOPT_CAINFO => 'i:/cacert.pem',
```

Cette option est de nouveau obligatoire avec l'API GitHub.

Vous pouvez obtenir ce certificat depuis la source fiable : https://curl.se/docs/caextract.html
Rappel : Le certificat SSL joue un rôle essentiel dans le protocole HTTPS en garantissant le chiffrement des données, l'authentification du serveur, l'intégrité des données, et en ajoutant une couche de confiance et de sécurité

3. La transmission des données par la méthode POST

Il existe différentes façons d'envoyer des données via POST.

form-data (multipart/form-data)

Principalement utilisé pour envoyer des fichiers (upload) et des données de formulaire
Permet l'envoi de fichiers binaires
Format similaire à celui d'un formulaire HTML classique

Dans le code de l'API, la récupération des données envoyées s'effectue en utilisant la variable superglobale $_POST[]

```
$nom = $_POST['nom'];
$fichier = $_FILES['monFichier'];
```

x-www-form-urlencoded

Format par défaut des formulaires HTML
Les données sont encodées comme une chaîne de requête URL
Format : cle1=valeur1&cle2=valeur2
Ce format est à utiliser si vous devez transmettre des données simples (texte)

La récupération des données envoyées s'effectue en utilisant la variable superglobale $_POST[]

raw (données brutes)

Permet d'envoyer des données dans n'importe quel format (JSON, XML, texte, etc.)
Couramment utilisé pour envoyer du JSON dans les API REST

La récupération des données envoyées s'effectue de la façon suivante pour le format JSON

```
$donnees = json_decode(file_get_contents('php://input'), true);
$nom = $donnees['nom'];
$age = $donnees['age'];
```

Selon la méthode utilisée, l'entête 'CONTENT_TYPE' de la requête prend une valeur différente :

Méthode	Valeur de l'entête 'CONTENT_TYPE'
Raw (json)	**'application/json'**
x-www-form-urlencoded	**'application/x-www-form-urlencoded'**
form-data (multipart/form-data)	**'multipart/form-data'**

Au nveau de l'API, la valeur se récupère à l'aide de la variable super globale $_SERVER['CONTENT_TYPE']

En règle générale, c'est l'API qui dans sa documentation précise et donc impose la méthode à employer pour lui envoyer des données.

```
Méthode : POST
Url : /api/calendrier/

Content-Type: application/json

Corps de la requête (JSON):
{
   "numero": "string",
   "nom": "string",
   "date": "YYYY-MM-DD",
   "distance": "string"
}

Réponses possibles :
- 201: Course créée
- 400: Données invalides
- 415: Format de contenu non supporté
```

4. La réponse de l'API

La réponse est le plus souvent retournée dans le format JSON.

Il existe des standards et des bonnes pratiques pour la structure de la réponse au format JSON d'une API, bien que ces standards ne soient pas toujours strictement définis et peuvent varier en fonction des besoins spécifiques de l'application.

Voici une structure de réponse JSON possible :

```
{
   "message": "Description de la réponse ou de l'erreur",
   "data": {
      // Données de la réponse, si applicable
   },
   "errors": [
      {
         "id": "Description de l'erreur concernant ce champ",
         …
        "global" : "Description d'une erreur globale"
      }
   ]
}
```

La propriété data contient l'élément ajouté ou les modifications enregistrées.
La propriété errors permet de détailler chaque erreur détectée.

Pour récupérer la réponse :

```
$reponseJson = curl_exec($ch);
```

5. Le code de statut http associé à la réponse

Cette réponse s'accompagne d'un code de statut http (200, 201, 204, 400 415, 422, etc.). Ce code permet déjà de savoir si le message contenu dans la réponse de l'API est un message de succès (codes http 200) ou d'erreur (codes http 300 et 400)

Le code de statut HTTP peut être récupéré en PHP par la fonction curl_getinfo.

```
$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
```

Dans l'API, il est renseigné à l'aide de la fonction http_response_code(valeur);

```
http_response_code(200);
```

6. Les vérifications

Il faut commencer par vérifier que la requête http s'est exécutée correctement :

```
if (curl_errno($ch)) {
    curl_close($ch);
    Erreur::envoyerReponse("Erreur cURL: " . curl_error($ch), 'global');
}
```

Ensuite il faut vérifier le code de statut HTTP et selon ce code, inspecter le contenu de la réponse pour connaitre la raison précise de l'erreur

Utiliser uniquement le code http est une solution simple mais elle manque de détail. La classe Erreur comporte une méthode qui retourne une description du code http :

```
Erreur::getErreurHttp($httpCode)
```

La vérification de la réponse permet d'obtenir des informations détaillées sur l'erreur, mais elle nécessite de parser la réponse et d'en connaitre sa structure qui varie d'une API à l'autre.

Exemple du code interrogeant une API réalisant une suppression

```
// Vérification des erreurs cURL
if (curl_errno($ch)) {
    curl_close($ch);
    Erreur::envoyerReponse("Erreur cURL: " . curl_error($ch), 'global');
}

// Décodage de la réponse JSON
$reponse = json_decode($reponseJson, true);

// récupération du message de la réponse
$msg = $reponse['message'];

// Récupération du code HTTP
$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
```

```
// Fermeture de la connexion cURL
curl_close($ch);

// analyse du code de retour
if ($httpCode === 200) {
    echo json_encode(['success' => $msg]);
} elseif ($httpCode === 400) {
    $msg .= " : " . $reponse['details'] ?? '';
    echo json_encode(['success' => $msg]);
} elseif ($httpCode === 401) {
    echo json_encode(['success' => $msg]);
} else {
    Erreur::envoyerReponse(Erreur::getErreurHttp($httpCode), 'global');
}
```

L'API utilise ici le code 200 pour pouvoir associer la réussite de la requête à un message de retour
Si l'API utilise le code 204 aucun message n'est associé.

7. Exemple interrogeant une API permettant la création d'un enregistrement

```
<?php
declare(strict_types=1);

// activation du chargement dynamique des ressources
require $_SERVER['DOCUMENT_ROOT'] . "/include/autoload.php";

// Vérifier les paramètres POST
if (!Std::existe('id', 'nom', 'date', 'distance')) {
    Erreur::envoyerReponse("Paramètres manquants", 'global');
}

// récupération des paramètres transmis
$distance = $_POST['distance'];
$nom = $_POST['nom'];
$date = $_POST['date'];
$id = $_POST['id'];

// Récupérer le token d'autorisation depuis la variable d'environnement
$token = getenv('CALENDRIER_TOKEN');

// Initialisation de cURL
$url = "http://api/calendrier/";
$ch = curl_init();

// Données à envoyer à l'API GitHub
$data = [
    'id' => $id,
    'nom' => $nom,
    'date' => $date,
    'distance' => $distance
];
```

```
// Configuration des options cURL
$options = [
    CURLOPT_URL => $url,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_TIMEOUT => 30,
    CURLOPT_HTTPHEADER => [
        'Content-type: application/json',
        'Authorization: Bearer ' . $token
    ],
    CURLOPT_POST => true,
    CURLOPT_POSTFIELDS => json_encode($data)
];
curl_setopt_array($ch, $options);

// Exécution de la requête
$reponseJson = curl_exec($ch);

// Vérification des erreurs cURL
if (curl_errno($ch)) {
    curl_close($ch);
    Erreur::envoyerReponse("Erreur cURL: " . curl_error($ch), 'global');
}

// Récupération du code HTTP
$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);

// Fermeture de la connexion cURL
curl_close($ch);

// Décodage de la réponse JSON
$reponse = json_decode($reponseJson, true);

// récupération du message de la réponse
$msg = $reponse['message'];

// Vérification du code HTTP
if ($httpCode === 201) {
    echo json_encode(['success' => $msg]);
} elseif ($httpCode === 400) {
    if (isset($reponse['details'])) {
        echo json_encode(['error' => $reponse['details']]);
    } else {
        Erreur::envoyerReponse($msg, 'global');
    }
} elseif ($httpCode === 401) {
    Erreur::envoyerReponse($msg, 'global');
} else {
    Erreur::envoyerReponse(Erreur::getErreurHttp($httpCode), 'global');
}
```